PROGRAM 14

AIM

To implement the Alpha-Beta Pruning algorithm for the Connect Four game to find the best possible move by reducing unnecessary game-state evaluations.

ALGORITHM

1. Start the program.
2. Create the Connect Four game board.
3. Define the possible moves and winning conditions.
4. Set the initial values of alpha and beta.
5. Generate possible moves for the current player.
6. Evaluate the game state using the Minimax approach.
7. Update the alpha value for the maximizing player.
8. Update the beta value for the minimizing player.
9. If alpha becomes greater than or equal to beta, prune the remaining branches.
10. Select the move with the best evaluated score.
11. Display the selected move and game result.
12. Stop the program.

PROGRAM

```
ROWS = 4
        for c in range(COLS - 3):
            if all(board[r][c+i] == player for i in range(4)):
                return True
    for r in range(ROWS - 3):
        for c in range(COLS):
            if all(board[r+i][c] == player for i in range(4)):
                return True
    for r in range(ROWS - 3):
        for c in range(COLS - 3):
            if all(board[r+i][c+i] == player for i in range(4)):
                return True
    for r in range(3, ROWS):
```

```
        for c in range(COLS - 3):
            if all(board[r-i][c+i] == player for i in range(4)):
                return True
    return False
def alphabeta(depth, alpha, beta, maximizing):
    if winner('O'):
        return 10
    if winner('X'):
        return -10
    if depth == 0 or not valid_moves():
        return 0
    if maximizing:
        value = -float('inf')
        for col in valid_moves():
            row = make_move(col, 'O')
            value = max(value, alphabeta(depth - 1, alpha, beta, False))
            undo_move(row, col)
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value
    else:
        value = float('inf')
        for col in valid_moves():
            row = make_move(col, 'X')
            value = min(value, alphabeta(depth - 1, alpha, beta, True))
            undo_move(row, col)
            beta = min(beta, value)
            if alpha >= beta:
                break
        return value
def best_move():
```

```
    best_score = -float('inf')
    move = valid_moves()[0]
    for col in valid_moves():
        row = make_move(col, 'O')
        score = alphabeta(4, -float('inf'), float('inf'), False)
        undo_move(row, col)
        if score > best_score:
            best_score = score
            move = col
    return move
print("Connect Four using Alpha-Beta Pruning")
display()
while True:
    col = int(input("Enter column (1-4): ")) - 1
    if col not in valid_moves():
        print("Invalid move")
        continue
    make_move(col, 'X')
    display()
    if winner('X'):
        print("Player X wins!")
        break
    if not valid_moves():
        print("Game Draw!")
        break
    computer = best_move()
    make_move(computer, 'O')
    print("Computer chooses column:", computer + 1)
    display()
    if winner('O'):
        print("Computer O wins!")
        break
```

OUTPUT

```
Connect Four using Alpha-Beta Pruning
 |  |  |
-------
 |  |  |
-------
 |  |  |
-------
 |  |  |
-------
Enter column (1-4): 1
 |  |  |
-------
 |  |  |
-------
 |  |  |
-------
X|  |  |
-------
Computer chooses column: 1
 |  |  |
-------
 |  |  |
-------
O|  |  |
-------
X|  |  |
-------
Enter column (1-4): 2
 |  |  |
-------
 |  |  |
-------
O|  |  |
-------
X|X|  |
-------
Computer chooses column: 1
 |  |  |
-------
O|  |  |
-------
O|  |  |
-------
X|X|  |
-------
```

RESULT

The Alpha-Beta Pruning algorithm was successfully implemented for the Connect Four game. The algorithm reduced unnecessary game-state evaluations while selecting suitable moves for the computer.